Informe Técnico Integrador · Segunda Entrega

Proyecto «Food Store» · Persistencia Políglota y Gestión de Datos (U4 a U8)

Institución: Universidad Tecnológica Nacional (UTN) · Tecnicatura Universitaria en Programación
Asignatura: Base de Datos II · Ciclo Lectivo 2026
Integrantes del Grupo: Máximo Monardez · Thiago Oviedo · Matías Ruiz
Motores: PostgreSQL 17 (OLTP) · MongoDB 9.0+ (NoSQL Read Model) · SQLite 3.37+ (Edge POS)
Repositorio: https://github.com/TekkNo0/BDTPI | **Rama de Entrega:** unidades-7-8

---

1. Arquitectura Políglota de «Food Store»

El proyecto integrador Food Store articula una solución de datos distribuida y políglota para un modelo de negocio mixto (restaurante / food truck en ferias gastronómicas):

Motor	Rol Arquitectónico	Responsabilidad y Datos Custodiados	Soberanía
PostgreSQL 17	Núcleo Transaccional (OLTP / ACID)	Catálogo maestro, inventario real, facturación firme, auditoría de seguridad y roles.	**SSOT (Única Fuente de Verdad)**
MongoDB 9.0+	Modelo Documental (Read Model)	Colección desnormalizada de pedidos y líneas embebidas para analítica rápida sin JOINs.	Solo Lectura / Sincronizado
SQLite 3.37+	Caché Periférica (Offline-First)	Instantánea local del catálogo en tablet POS y borradores de pedidos generados en feria.	El móvil propone, el servidor dispone

2. Elementos Implementados por Unidad (4 a 8)

• **Unidad 4 (Normalización y Redundancia):** Detección de anomalías en control de lotes y refactorización a **Forma Normal de Boyce-Codd (FNBC)**. Migración de datos sin pérdida en transacciones reversibles. Desnormalización controlada con tabla resumen venta_categoria_diaria mantenida mediante triggers automáticos y validada con EXPLAIN ANALYZE.
• **Unidad 5 (Seguridad y Auditoría):** Implementación del principio de mínimo privilegio con 4 roles (foodstore_admin, foodstore_operador, foodstore_auditor, rol_cajero). Auditoría nativa de accesos y DDL. Simulacro de acceso indebido/escalamiento de privilegios reconstruido por logs forenses. Anonimización con hashes criptográficos SHA-256 en usuario_anon.
• **Unidad 6 (Mantenimiento y Recuperación):** Respaldo lógico automatizado en formato binario custom (pg_dump -Fc) vía scripts en PowerShell y Bash. Rutinas de mantenimiento seguro (maintenance.sql y health_check.sql sin VACUUM FULL). Simulacro de desastre con restauración contra reloj (RTO = 10.49 s, RPO = 0 s) e informe post-incidente.
• **Unidad 7 (NoSQL Documental):** Pipeline ETL en Python (migrar_datos.py) para migrar pedidos de PostgreSQL a MongoDB embebiendo cliente y detalle. Batería de 4 consultas analíticas en mongosh y consultas equivalentes en SQL con análisis comparativo.
• **Unidad 8 (Bases Móviles e Integración):** Base SQLite foodstore_movil.db en tablet POS con directiva STRICT y patrón Money (centavos enteros). Algoritmo del Semáforo (semaforo.sql) y simulador en Python (simulador.py) para resolución determinista de conflictos C2 a C6. Code Review de toda la cursada asistido por IA.

3. Metodología de Pruebas y Resultados Cuantitativos

Desempeño de Desnormalización (U4): En un dataset poblado con 200.000 pedidos y 600.000 líneas, la consulta normalizada con agregación y múltiples JOINs demandó **185.34 ms**. La consulta sobre venta_categoria_diaria resolvió en **0.82 ms** (aceleración superior a **220x**) con actualización atómica continua garantizada por triggers.
Simulacro de Intrusión (U5): Se comprobó el bloqueo inmediato del rol cajero al intentar alterar precios (ERROR: permission denied for table producto). El registro de auditoría capturó IP, usuario, hora e instrucción exacta del atacante permitiendo diseñar la contención preventiva.
Simulacro de Desastre (U6): Ante la pérdida simulada del servidor, el script desatendido restore_test.ps1 aprovisionó la base limpia, restauró datos, índices y restricciones en **10.49 segundos (RTO)** con **0 segundos de pérdida (RPO)**, validado por validaciones_minimas.sql.
Benchmark PostgreSQL vs. MongoDB (U7): La lectura de pedidos con detalle completo en MongoDB (findOne) tomó **1.2 ms** sin sobrecosto de JOINs, frente a **4.8 ms** en PostgreSQL. PostgreSQL demostró mayor eficiencia en agregaciones complejas sobre tablas altamente indexadas.
Casos de Conflicto de Sincronización Móvil (U8): El simulador evaluó los 5 escenarios deterministas: C2 (Aprobado por stock/precio idéntico), C3 (Rechazado por stock insuficiente), C4 (Conflicto por incremento de precio en servidor → semáforo amarillo para re-cotización), C5 (Rechazado por ítem inactivo/lápida) y C6 (Rechazado integral por falla en una línea de pedido mixto).

4. Decisiones Fundamentadas de Diseño y Arquitectura

• **FNBC vs. Desnormalización:** Se normalizó a FNBC en entidades donde las dependencias transitivas generaban anomalías de modificación (lotes y depósitos). Se desnormalizó selectivamente en reportes de alta frecuencia donde el cálculo dinámico penalizaba el CPU, blindando la consistencia mediante triggers transaccionales.
• **Mínimo Privilegio:** El personal de punto de venta (cajeros) carece de permisos de DDL o de modificación sobre transacciones cerradas, garantizando inmutabilidad contable.
• **Formato de Respaldo:** Se eligió pg_dump -Fc frente a volcados en texto plano (SQL) por comprimir el almacenamiento en más del 65% y permitir restauración paralela (-j) y selectiva.
• **Regla de Oro en Movilidad:** *'El móvil propone, el servidor dispone'*. La tablet no confirma pedidos ni altera inventario; genera borradores que PostgreSQL valida y arbitra de forma definitiva.
• **Patrón Money en SQLite:** Se descartó el tipo REAL debido a los errores de redondeo inherentes a la coma flotante IEEE 754; se adoptó INTEGER en centavos enteros con tabla STRICT.

5. Declaración DUIA y Code Review Asistido por IA

Se aplicó el Protocolo de IA sobre datos: anonimización absoluta de información y revisión humana crítica de cada propuesta. A continuación se resumen los hallazgos del Code Review sobre los scripts de la cursada:

#	Unidad / Script	Propuesta de la IA	Decisión	Fundamentación Técnica
H 1	U1 / schema.sql	Reemplazar CHECK (precio >= 0) por triggers PL/pgSQL.	**DESCARTADO**	Las restricciones CHECK declarativas corren en el kernel de C sin latencia ni sobrecosto.
H 2	U5 / roles.sql	Permisos excesivos (GRANT ALL) en rol cajero.	**ACEPTADO**	Se corrigió a mínimo privilegio: solo SELECT, INSERT revocando UPDATE.
H 3	U8 / schema_sqlite.sql	Almacenar importes en SQLite como REAL con decimales.	**DESCARTADO**	Evita errores de redondeo binario. Se aplicó patrón Money en centavos enteros (INTEGER).
H 4	U7 / migrar_datos.py	Cargar pedidos en MongoDB con bucle insertOne.	**DESCARTADO**	Ineficiente por latencia RTT. Se usó insertMany ejecutando el lote en < 10 ms.

6. Checklist de Cumplimiento Oficial de Entrega

[X] **Unidad 4:** Normalización a FNBC validada, migración reversible y desnormalización con triggers.
[X] **Unidad 5:** Mínimo privilegio, auditoría activada, simulacro de intrusión y tabla anonimizada.
[X] **Unidad 6:** Respaldo programado, rutinas de salud y simulacro de desastre con informe post-incidente.
[X] **Unidad 7:** Base documental MongoDB sobre datos reales y comparativa técnica vs. relacional.
[X] **Unidad 8:** SQLite móvil offline-first, algoritmo de semáforo con conflictos probados y Code Review.
[X] **Criterio Cátedra:** Repositorio único con branch unidades-7-8 e Informe Técnico Integrador.